

Flask - Day 10

Email Communication & OTP Verification

SNS Project Milestone | PFS-HYD-054

Register → **Generate OTP** → **Send Email** → **Verify** → **Activate**

Today students turn registration into account verification.

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's Learning Outcomes

By the end, students should explain the full OTP lifecycle.



1. Explain why email verification is added after registration
2. Generate a short-lived OTP using a secure random source
3. Store OTP safely using hash + expiry instead of plain text
4. Send OTP through SMTP using a separate email helper
5. Verify OTP and activate the user account
6. Test success, invalid OTP, expired OTP and email failures

Day 9 Recap → Day 10 Upgrade

Registration stored users; today we decide whether the user is verified.



Day 9 completed

- Registration form
- request.form handling
- Server-side validation
- Duplicate email check
- bcrypt password hash
- INSERT into users

Day 10 adds

- OTP generation
- Email delivery
- OTP hash + expiry
- Verify OTP form
- s_verified = TRUE
- Clear OTP after success

Why Email Verification?

Registration alone proves that a form was submitted, not that the email belongs to the user.



Problem

Fake email addresses can be registered. A user may mistype email.
Account recovery later becomes unreliable.

Solution

Send a one-time password to the email. Activate the account only when the correct OTP is submitted before expiry.

Key idea: verification connects identity input with real inbox access

Day 10 Core Feature Flow

This is the board diagram students must be able to draw.



Registration → OTP generation → Hash + expiry → Email → Verification → Activation

1. Plain OTP is emailed once
2. Database stores otp_hash and otp_expires_at
3. Verify route checks submitted OTP against the stored hash
4. If valid and not expired, set is_verified = TRUE
5. Then clear otp_hash and otp_expires_at

Database Columns Needed

Day 10 depends on the users table design from Day 6.



```
CREATE TABLE users (  
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  is_verified BOOLEAN NOT NULL DEFAULT FALSE,  
  otp_hash VARCHAR(255) NULL,  
  otp_expires_at DATETIME NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Column meaning

s_verified: account activation status

otp_hash: hashed OTP, not plain OTP

otp_expires_at: last valid time for OTP

Why nullable?

After successful verification, OTP data should be cleared because it is no longer needed.

Generating a Six-Digit OTP

Use a security-appropriate random source for verification secrets.



```
import secrets
from datetime import datetime, timedelta

def generate_otp():
    otp = f"{secrets.randbelow(1_000_000):06d}"
    expires_at = datetime.utcnow() + timedelta(minutes=10)
    return otp, expires_at
```

Why this format?

`secrets.randbelow(1_000_000)` returns a number from 0 to 999999. The `:06d` formatting keeps leading zeroes.

Examples

000421
097532
784201
999999

Do Not Store the Plain OTP

If the database leaks, plain OTPs should not be readable.



Bad practice

- otp = 123456 stored directly
- Anyone with DB access can read it
- OTP may be accidentally logged
- Difficult to justify as secure

Better practice

- Email plain OTP once
- Store bcrypt OTP hash
- Verify using `check_password_hash`
- Clear OTP after success

Hash OTP + Expiry

Create a short-lived secret and store the safe representation.



```
otp, expires_at = generate_otp()
otp_hash = bcrypt.generate_password_hash(otp).decode("utf-8")

cursor.execute(
    """
    UPDATE users
    SET otp_hash = %s,
        otp_expires_at = %s
    WHERE email = %s
    """
    ,
    (otp_hash, expires_at, email)
)
connection.commit()
```

Teaching point

The database receives otp_hash and otp_expires_at. The plain OTP should only be used to send the email, then forgotten.

Validation later

Submitted OTP must match the stored hash AND current time must be before expiry.

Email Communication Flow

Flask does not deliver email directly to inboxes; it submits email through an SMTP provider.



Flask app → SMTP provider → Recipient mail server → Inbox

1. Create message: subject, sender, recipient and body
2. Connect to SMTP server securely
3. Authenticate using app password or provider credential
4. Submit the message
5. Handle errors without exposing credentials

SMTP, smtplib and EmailMessage

Three terms students must recognize before coding.



SMTP

Simple Mail Transfer Protocol. Used to transfer email between applications and mail servers.

smtplib

Python standard library module that connects to an SMTP server and sends the email.

EmailMessage

Python object used to build subject, sender, recipient and email content cleanly.

Secure Email Configuration

Credentials must not be hardcoded into app.py or committed to GitHub.



```
.env
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your_email@gmail.com
MAIL_PASSWORD=your_app_password
MAIL_FROM=your_email@gmail.com
```

```
.gitignore
venv/
__pycache__/
.env
*.pyc
```

Classroom warning

Never show real app passwords on projector. Use placeholders or a demo-only account.

Replaceable infrastructure

Provider policies can change. Keep email settings separate so the app can switch providers later.

Create mail_service.py

Modularization keeps email logic separate from route logic.



```
# mail_service.py
import smtplib
from email.message import EmailMessage

def send_otp_email(to_email, otp):
    message = EmailMessage()
    message["Subject"] = "SNS Account Verification OTP"
    message["From"] = "your_email@gmail.com"
    message["To"] = to_email
    message.set_content(f"Your SNS OTP is {otp}. It expires in 10 minutes.")

    with smtplib.SMTP("smtp.gmail.com", 587) as server:
        server.starttls()
        server.login("your_email@gmail.com", "app_password")
        server.send_message(message)
```

Why modular?

The register route should register. The mail helper should send mail. Small functions are easier to test and debug.

Later improvement

Move credentials into environment variables and add exception handling.

Registration Route: What Changes Today?

Day 9 inserted user; Day 10 also creates and sends OTP.



```
# After validation and password_hash creation
otp, expires_at = generate_otp()
otp_hash = bcrypt.generate_password_hash(otp).decode("utf-8")

cursor.execute(
    """
    INSERT INTO users
    (name, email, password_hash, otp_hash, otp_expires_at)
    VALUES (%s, %s, %s, %s, %s)
    """
    ,
    (name, email, password_hash, otp_hash, expires_at)
)
connection.commit()

send_otp_email(email, otp)
return redirect(url_for("verify_otp", email=email))
```

1. Create user with is_verified = FALSE
2. Save OTP hash + expiry
3. Email plain OTP once
4. Redirect to OTP verification page

Verify OTP Route: GET and POST

GET displays the verification page; POST validates the submitted OTP.



```
@app.route("/verify-otp", methods=["GET", "POST"])
def verify_otp():
    email = request.args.get("email", "").strip().lower()

    if request.method == "GET":
        return render_template("verify_otp.html", email=email)

    submitted_otp = request.form.get("otp", "").strip()
    # next: lookup user, check expiry, verify hash, activate account
```

Why pass email?

The verification page needs to know which account is being activated. Later this can be improved with session or signed tokens.

Teaching point

The browser input is untrusted. Always normalize and validate submitted OTP.

Verification Logic

Correct OTP is not enough; it must also be unexpired and unused.



```
cursor.execute(
    "SELECT id, otp_hash, otp_expires_at FROM users WHERE email = %s",
    (email,)
)
user = cursor.fetchone()

if not user:
    return render_template("verify_otp.html", error="Invalid request")

if datetime.utcnow() > user["otp_expires_at"]:
    return render_template("verify_otp.html", error="OTP expired")

valid = bcrypt.check_password_hash(user["otp_hash"], submitted_otp)
if not valid:
    return render_template("verify_otp.html", error="Invalid OTP")
```

Check order

1. User exists
2. OTP exists
3. Not expired
4. Hash matches
5. Activate account

Test correct, wrong, expired and already-used OTP cases

Account Activation

After successful verification, activate the account and clear OTP fields.



```
cursor.execute(
    """
    UPDATE users
    SET is_verified = TRUE,
        otp_hash = NULL,
        otp_expires_at = NULL
    WHERE id = %s
    """
    (user["id"],)
)
connection.commit()

flash("Account verified. Please login.", "success")
return redirect(url_for("login"))
```

**Valid OTP → Activate → Clear
OTP → Login**

Why clear OTP?

A used OTP should not remain valid. Clearing it reduces accidental reuse and makes account state cleaner.

Optional: Resend OTP

Useful for real projects, but should not allow unlimited guessing or spam.



Resend flow

- Find unverified user
- Generate new OTP
- Replace old otp_hash
- Replace expiry
- Send fresh email

Safety checks

- Only unverified accounts
- Rate-limit resends
- Do not reveal account existence
- Log failures safely
- Avoid unlimited email attempts

Failure Cases Students Must Test

Day 10 is not complete until failure paths are demonstrated.



1. Email not received - check provider auth, spam, recipient and logged exception
2. OTP always invalid - normalize submitted value and verify against stored hash
3. Expired OTP accepted - check expiry before activation
4. Already-used OTP works again - clear OTP after successful verification
5. Plain OTP appears in logs - remove debug prints immediately

Testing Matrix

Every student should submit evidence for at least one success and two failures.



Valid OTP

Action: submit before expiry

Expected: account verified and redirected to login

Wrong OTP

Action: submit random OTP

Expected: error shown; account remains unverified

Expired OTP

Action: submit after expiry

Expected: rejected; no activation

Email failure

Action: invalid SMTP config

Expected: friendly error/log; no secret exposed

Reuse OTP

Action: submit OTP again after success

Expected: rejected because OTP was cleared

DB evidence

Show users.is_verified before and after verification, plus cleared OTP fields.

Trainer check: students must show database state before and after verification.

Live Debugging Exercise

Introduce one defect deliberately; students diagnose the failing layer.



Defect A

Store plain OTP, but verify using
`bcrypt.check_password_hash`.
Expected symptom: OTP always invalid.

Defect B

Forget to check `otp_expires_at`.
Expected symptom: expired OTP still activates
account.

Defect C

Use wrong SMTP password.
Expected symptom: email send fails; log shows
auth error.

Debug method: reproduce → inspect logs → identify layer → fix → retest

Security Best Practices for OTP

A six-digit OTP is not strong by itself; safety comes from surrounding controls.



1. Use secrets, not predictable random values, for OTP generation
2. Hash OTP before storing it when practical
3. Attach short expiry, such as 10 minutes
4. Clear OTP immediately after successful verification
5. Do not log OTP or email credentials
6. Add resend limits and attempt limits in production
7. Keep credentials in `.env` and `.env` inside `.gitignore`

Guided Lab: Day 10 Milestone

Students first reproduce the trainer version, then make one independent improvement.



1. Add OTP fields to users table if missing
2. Create generate_otp() helper
3. Create mail_service.py with send_otp_email()
4. Update registration flow to save OTP hash + expiry
5. Create verify_otp.html and verify_otp route
6. Activate account on correct OTP
7. Capture evidence for success and failures

1. Independent improvement options:
2. Cleaner validation messages
3. Resend OTP route
4. Better empty/error states
5. Database screenshots
6. Manual test case table in README

Day-End Assignment

Commit a stable Day-10 milestone before the next class.



1. Complete email communication and OTP verification
2. Show one successful account verification
3. Show two unsuccessful cases: wrong OTP and expired OTP
4. Update README with new routes and configuration names
5. Do not commit .env or real credentials
6. Commit message: Day 10: implement email communication & OTP verification

Submission: GitHub link + screenshots + short test evidence

Quick Quiz & Recap

Use this as an exit ticket.



1. Why should we not store a plain OTP?
2. What is the purpose of `otp_expires_at`?
3. What does SMTP do?
4. Why should email credentials be outside `app.py`?
5. What should happen after successful OTP verification?
6. Name two failure cases that must be tested.

Next class

Day 11: Login & Session Management

Credentials → user lookup → bcrypt verify → session creation → protected dashboard → logout